

GitHub Skills 第 5 节课柴建铭 2023250361021

柴建铭

2026 年 5 月

GitHub Skills 第 5 节课柴建铭 2023250361021

课程名称: Resolve merge conflicts

报告主题: 合并冲突产生原因与解决流程

学生姓名: 柴建铭

学号: 2023250361021

一、核心知识点梳理

本节课学习合并冲突的产生原因和解决方法。合并冲突通常发生在两个分支修改了同一文件的同一位置，Git 无法自动判断应该保留哪一部分内容时。冲突并不代表代码一定错误，而是需要开发者根据实际需求手动选择、整合或重写相关内容。

解决冲突的核心步骤包括: 识别冲突文件, 打开文件查看冲突标记, 理解 HEAD 和 incoming change 分别代表的内容, 删除冲突标记, 保留正确内容, 重新提交解决后的文件。在 GitHub 网页端或本地编辑器中都可以处理冲突, 但无论使用哪种方式, 都要确保最终文件没有残留冲突标记, 并且逻辑内容是完整的。

二、学习中的难点

学习中的难点是理解冲突标记的含义。文件中出现 «««<、=====、»»»> 时, 初学者容易误以为这些符号是系统错误, 实际上它们是 Git 用来提示冲突范围的标记。只有理解各部分代表当前分支和目标分支的不同内容, 才能正确处理。

另一个难点是解决冲突后仍然要进行检查。手动合并时可能出现重复内容、删除必要代码、语句不通顺等问题。如果只是机械地选择一边内容, 可能会让另一边的重要修改丢失。因此解决冲突不仅是删除标记, 更是重新判断两边修改的意图。

三、实践操作与收获

实践过程中，我按照课程引导进入存在冲突的 Pull Request，查看 GitHub 提示的冲突文件。打开冲突内容后，我先观察两边修改差异，再根据上下文判断应该保留哪部分内容，必要时把两边内容进行整合。确认最终文本完整后，删除冲突标记并提交解决结果，随后返回 Pull Request 页面确认冲突状态消失。

通过这次实践，我理解了为什么多人协作需要及时同步主分支。如果长期在个人分支中开发而不合并 main 分支的最新变化，最终合并时就更容易出现冲突。以后在项目中，我会尽量让每个分支只解决一个明确任务，并在提交 PR 前先检查是否与主分支存在差异，减少大范围冲突。

四、学习遇到的困惑与疑问（选做）

目前的疑问是：如果冲突涉及核心代码逻辑，而两边修改看起来都有必要，是否应该由原作者共同讨论后再解决？在大型项目中，是否有自动化测试帮助判断冲突解决后的代码是否仍然正确？

五、报告小结

通过本节 GitHub Skills 课程学习，我不仅完成了平台提供的实践任务，也进一步理解了软件工程课程中“版本控制、协作开发、文档规范、流程管理”的实际意义。后续学习中，我会继续按照规范提交内容，保持报告结构完整、表达准确、实践过程具体，并在项目训练中主动使用 GitHub 记录每一次关键修改。